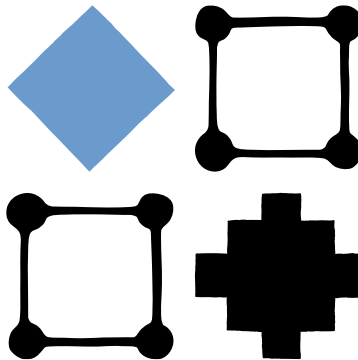


面向长时间应用开发的运行框架设计

Harness design for long-running application development

Anthropic · 2026-03-24 · 原文 / Source

完整译文与英文逐段对照。依据你提供的网页存档翻译，保留原文的观点、例子与限定条件。图表（含图注）、代码及其中的文字维持原文；网页导航、订阅表单和站点页脚不属于文章正文。中文为学习用非官方译文。



在 Agent 编程的能力前沿，运行框架设计是决定表现的关键。本文介绍我们如何进一步提升 Claude 在前端设计和长时间自主软件工程任务中的能力。

作者为我们 Labs 团队成员 Prithvi Rajasekaran。

过去几个月，我一直在研究两个相互关联的问题：如何让 Claude 产出高质量的前端设计，以及如何让它在无人干预的情况下构建完整应用。这项工作源于此前对前端设计技能和长时间运行的编程 Agent 运行框架的探索。我和同事通过提示工程和运行框架设计，将 Claude 的表现提升到了远高于基线的水平，但两个方向最终都遇到了瓶颈。

为了突破瓶颈，我开始寻找能够同时适用于两个迥异领域的新 AI 工程方法：一个领域依赖主观审美，另一个领域依赖可验证的正确性和可用性。受到生成对抗网络（GAN）的启发，我设计了一种包含生成器和评估器 Agent 的多 Agent 结构。要构建一个既能可靠评分、又有审美判断力的评估器，首先需要制定一组标准，将“这个设计好吗？”之类的主观判断，转化为具体、可评分的条目。

随后，我将这些技术应用于长时间自主编程，同时沿用了早期运行框架工作中的两点经验：把构建任务拆成可处理的小块，以及使用结构化产物在会话之间交接上下文。最终形成了一种由规划器、生成器和评估器组成的三 Agent 架构，能够在持续数小时的自主编程会话中，产出功能丰富的全栈应用。

简单实现为何不够

我们此前已经证明，运行框架设计会显著影响长时间 Agent 编程的有效性。在较早的一次实验中，我们使用初始化 Agent 将产品规格拆成任务列表，再由编程 Agent 每次实现一项功能，并在会话结束前交接产物，以便跨会话传递上下文。更广泛的开发者社区也逐渐形成了类似认识，例如“Ralph Wiggum”方法通过钩子或脚本，让 Agent 持续处于迭代循环中。

但有些问题始终存在。对于更复杂的任务，Agent 仍然容易随着时间推移而偏离正轨。在拆解这一问题时，我们观察到 Agent 执行此类任务时的两种常见失败模式。

第一，随着上下文窗口逐渐填满，模型在长任务中往往会丧失连贯性（参见我们的上下文工程文章）。有些模型还会表现出“上下文焦虑”：当它们认为自己正在接近上下文上限时，就开始过早收尾。上下文重置可以同时解决这两个问题：彻底清空上下文窗口，启动一个全新的 Agent，同时通过结构化交接传递前一个 Agent 的状态和后续步骤。

这与上下文压缩不同。压缩会在原处对对话较早的部分进行总结，让同一个 Agent 基于缩短后的历史继续运行。虽然压缩保留了连续性，但并没有给 Agent 一个全新的起点，因此上下文焦虑仍可能存在。重置提供了这样的起点，代价是交接产物必须包含足够的状态，让下一个 Agent 能够顺畅接手。在早期测试中，我们发现 Claude Sonnet 4.5 的上下文焦虑相当明显，仅靠压缩不足以让它在长任务中表现出色，因此上下文重置成为运行框架设计的必要组成部分。它解决了核心问题，但也为每次运行增加了编排复杂度、token 开销和延迟。

第二个问题是自我评估，我们此前尚未解决它。当被要求评估自己产出的工作时，Agent 往往会信心十足地赞扬自己的成果，即便在人类看来，质量显然平平。这一问题在设计等主观任务中尤为突出，因为不存在类似可验证软件测试的二元检查。一个布局给人的感觉是精致还是千篇一律，需要判断；而 Agent 给自己的作品评分时，总是偏向正面评价。

然而，即使任务结果可以验证，Agent 有时仍会表现出糟糕的判断力，妨碍任务完成。事实证明，将做事的 Agent 与评价结果的 Agent 分开，是解决这一问题的有力手段。这种分离本身并不会立刻消除宽松倾向；评估器仍然是一个倾向于宽待大语言模型产物的大语言模型。但事实表明，调优一个独立评估器，让它持怀疑态度，比让生成器批判自己的工作容易得多。而一旦有了这种外部反馈，生成器就有了具体的迭代依据。

前端设计：让主观质量可以评分

我从前端设计开始实验，因为自我评估问题在这里最明显。如果不加干预，Claude 通常会偏向稳妥、可预测的布局：技术上能够使用，但视觉上乏善可陈。

两点认识塑造了我为前端设计构建的运行框架。首先，虽然美感不能完全归结为一个分数，而且个人品味永远会有差异，但通过纳入设计原则和偏好的评分标准，仍然可以改善美感。“这个设计漂亮吗？”很难得到一致回答，而“它符合我们对好设计的原则吗？”则为 Claude 提供了具体的评分依据。其次，将前端生成与前端评分分开，可以形成一个反馈循环，推动生成器产出更好的结果。

据此，我编写了四项评分标准，并在提示词中同时提供给生成器和评估器 Agent：

- **设计质量：**设计给人的感觉是一个协调的整体，还是各个部分的拼凑？优秀的表现意味着色彩、字体、布局、图像和其他细节共同营造出独特的氛围与辨识度。
- **原创性：**能否看出针对具体项目作出的设计决策，还是只用了模板布局、组件库默认设置和 AI 生成的套路？人类设计师应该能够辨认出有意为之的创意选择。未修改的现成组件，或白色卡片叠加紫色渐变等明显的 AI 生成特征，在这一项都不合格。
- **工艺：**技术执行层面，包括字体层级、间距一致性、色彩协调以及对比度。这是能力检查，而非创意检查。大多数合理实现默认都能做好；如果这一项不合格，就意味着基本功出了问题。
- **功能性：**独立于美感的可用性。用户是否能理解界面的用途、找到主要操作，并且无需猜测就能完成任务？

相比工艺和功能性，我更强调设计质量与原创性。Claude 默认就在工艺和功能性上得分不错，因为所需的技术能力通常是模型自然具备的。但在设计与原创性方面，Claude 的产出往往最多只能算平淡。这套标准明确惩罚高度套路化的“AI 垃圾内容”模式，并通过提高设计与原创性的权重，推动模型在审美上作出更大胆的尝试。

我使用带有详细分项评分的少样本示例来校准评估器。这确保了评估器的判断与我的偏好一致，也减少了不同迭代之间的评分漂移。

我基于 Claude Agent SDK 构建了这个循环，使编排保持简单。生成器 Agent 先根据用户提示创建 HTML/CSS/JS 前端。我为评估器提供了 Playwright MCP，让它在逐项评分并撰写详细评论之前，能够直接与运行中的页面交互。实际运行中，评估器会自主浏览页面、截图并仔细研究实现，然后给出评估。反馈会回传给生成器，作为下一轮迭代的输入。每次生成我会运行 5 至 15 轮迭代；通常，生成器在回应评估器的评论时，每一轮都会朝更有特色的方向推进。由于评估器实际在操作页面，而不是只给静态截图打分，每个循环都需要实打实的时间。完整运行最长会达到四小时。我还要求生成器在每次评估后作出策略决策：如果分数走势良好，就继续打磨当前方向；如果当前方案行不通，就转向完全不同的审美风格。

在多次运行中，评估器的评分会随着迭代提高，之后进入平台期，但仍有提升空间。有些生成结果是渐进式改进，有些则会在两轮迭代之间发生明显的审美转向。

评分标准的措辞对生成器的引导方式，有些出乎我的预料。例如，加入“最好的设计应具有博物馆级品质”这样的表述，会让设计趋向某种特定的视觉风格。这表明，围绕评分标准的提示语言，会直接塑造输出的特征。

虽然评分总体上会随迭代提高，但这一过程并不总是清晰的线性增长。后期实现整体上通常更好，但我也经常遇到更喜欢中间某一版、而不是最后一版的情况。实现复杂度也往往逐轮上升，因为生成器会根据评估器的反馈，尝试更有雄心的解决方案。即使在第一轮迭代，产出也已经明显优于完全不加这些提示的基线。这说明，在评估器反馈带来进一步打磨之前，评分标准及其措辞本身，就已使模型偏离了平庸的默认方案。

有一个特别值得一提的例子：我要求模型为一家荷兰美术馆创建网站。到了第九轮，它为一家虚构的美术馆生成了一个简洁的深色主题首页。页面在视觉上已经很精致，但大体仍在我的预期之内。到了第十轮，它却完全推翻了原有方案，将网站重新构想为一种空间体验：一个用 CSS 透视渲染棋盘格地板的 3D 房间，艺术作品自由分布在墙面上，画廊房间之间通过门口导航，而不是靠滚动或点击。这种创意跃迁，是我此前从未在一次性生成中见过的。

Video / 视频（联网播放）

扩展到全栈编程

有了这些发现，我将这一受 GAN 启发的模式应用于全栈开发。生成器—评估器循环与软件开发生命周期天然契合：代码审查和质量保证在结构上承担着与设计评估器相同的角色。

架构

在早期的长时间运行框架中，我们通过初始化 Agent、每次处理一项功能的编程 Agent，以及会话之间的上下文重置，解决了跨会话编程的连贯性问题。上下文重置是关键突破：当时框架使用 Sonnet 4.5，它会表现出前文提到的“上下文焦虑”。构建一个能在上下文重置后仍然良好运转的框架，是让模型始终围绕任务工作的关键。Opus 4.5 本身已基本消除了这种行为，因此我得以从这个框架中彻底移除上下文重置。各 Agent 在整个构建过程中以连续会话运行，由 Claude Agent SDK 的自动压缩机制处理不断增长的上下文。

这次，我在原有运行框架的基础上构建了一个三 Agent 系统，每个 Agent 都针对我在此前运行中观察到的特定缺口。系统包含以下 Agent 角色：

规划器：我们之前的长时间运行框架要求用户预先提供详细规格。我想将这一步自动化，因此创建了一个规划器 Agent：接收简单的 1 至 4 句话提示，并将其扩展为完整产品规格。我在提示中要求它在功能范围上有雄心，并聚焦产品背景和高层技术设计，而非详细技术实现。这样强调，是因为我担心：如果规划器试图预先指定细粒度技术细节却出了错，规格中的错误就会传导至下游实现。更明智的做法似乎是约束 Agent 必须交付什么，让它们在工作中自行决定实现路径。我还要求规划器寻找机会，将 AI 功能融入产品规格。（文末附录提供了一个示例。）

生成器：此前框架中一次处理一项功能的方法，在管理任务范围上效果很好。我在这里采用了类似模式，要求生成器以迭代周期（sprint）开展工作，每次从规格中领取一项功能。每个迭代周期都使用 React、Vite、FastAPI 和 SQLite（后来改为 PostgreSQL）技术栈来实现应用，并要求生成器在每轮结束时先自评，再交给质量保证环节。它还可以使用 git 进行版本控制。

评估器：早期运行框架生成的应用往往看起来令人印象深刻，但实际使用时仍会遇到实实在在的缺陷。为了发现这些问题，评估器使用 Playwright MCP，像用户一样逐步点击运行中的应用，测试界面功能、API 端点和数据库状态。随后，它会结合发现的缺陷，以及一组从前端实验调整而来的标准，对每个迭代周期评分；这些标准覆盖产品深度、功能性、视觉设计和代码质量。每项标准都有硬性阈值，只要有任何一项低于阈值，该迭代周期就不通过，生成器会收到关于问题的详细反馈。

每个迭代周期开始前，生成器和评估器都会协商一份迭代契约：在编写任何代码之前，先就这部分工作“完成”的含义达成一致。之所以这样做，是因为产品规格刻意保持在较高层次，我希望增加一个步骤，填补用户故事与可测试实现之间的空白。生成器提出要构建什么、如何验证成功；评估器审查提案，确保生成器构建的是正确的东西。双方持续迭代，直到达成一致。

通信通过文件进行：一个 Agent 写入文件，另一个 Agent 读取后，在同一文件中作答，或新建文件供前一个 Agent 再读取。随后，生成器按照约定的契约构建，再将成果交给质量保证环节。这既让工作忠于规格，也避免过早将实现方式规定得过细。

运行框架

在这一框架的首个版本中，我使用 Claude Opus 4.5，将相同用户提示分别交给完整框架和单 Agent 系统，以便比较。之所以选择 Opus 4.5，是因为它是我开始这些实验时最好的编程模型。

我编写了下面这条提示，用来生成一个复古电子游戏制作工具：

创建一个 2D 复古游戏制作工具，功能包括关卡编辑器、精灵编辑器、实体行为，以及可实际游玩的测试模式。

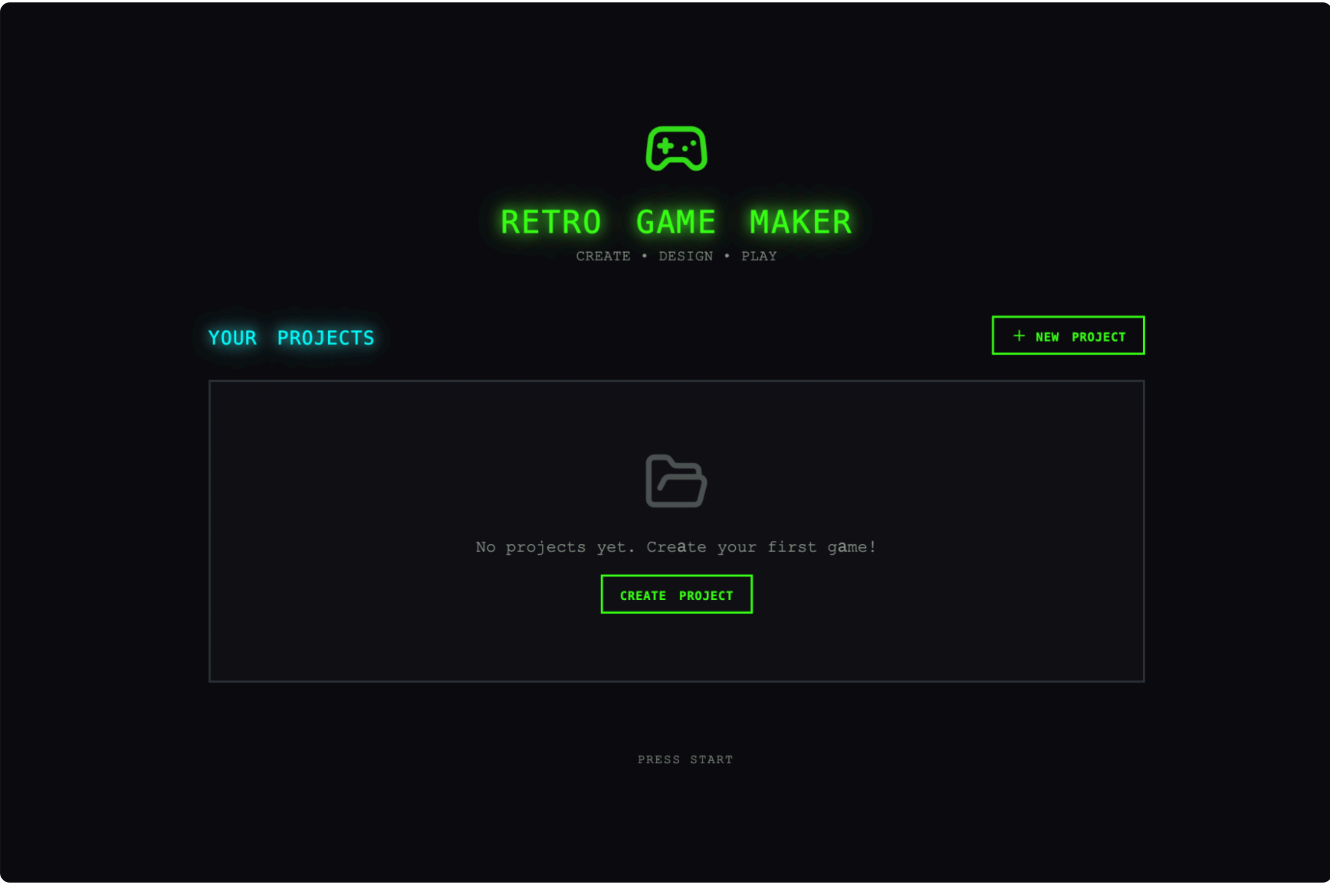
下表列出了运行框架类型、运行时长和总成本。

Harness	Duration	Cost
Solo	20 min	\$9
Full harness	6 hr	\$200

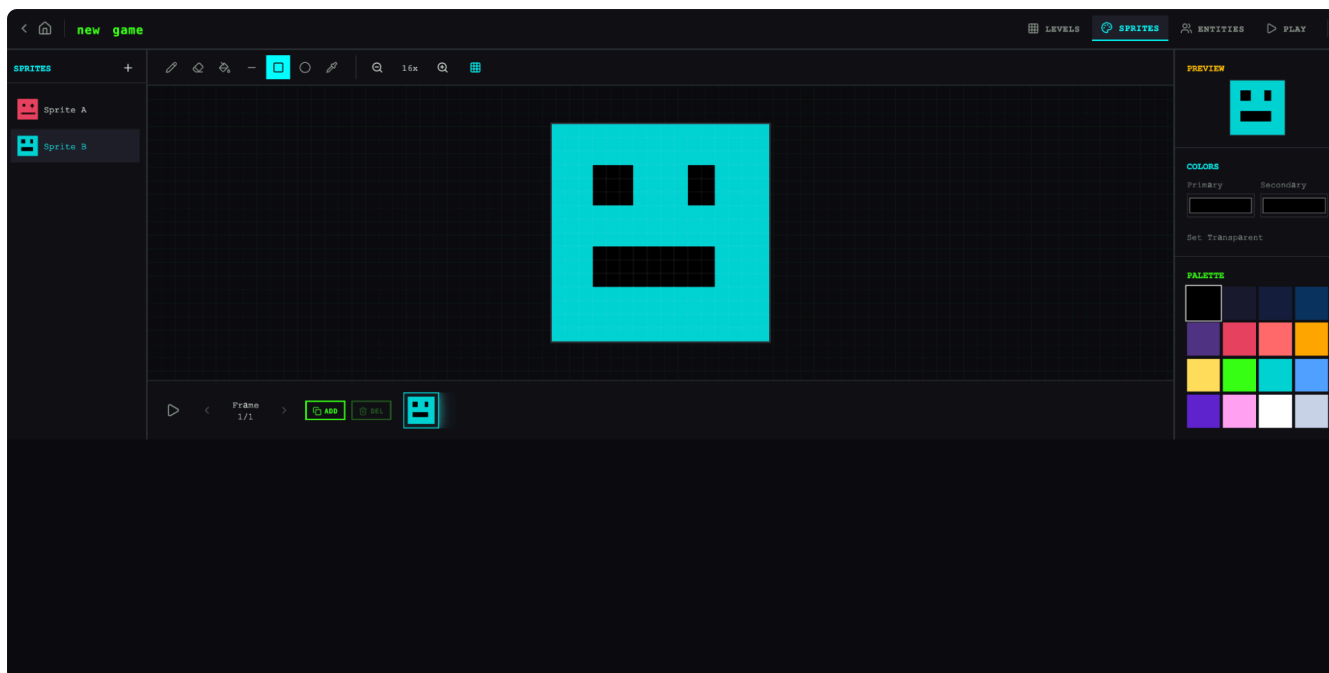
完整框架的成本高出二十多倍，但输出质量的差别一眼就能看出来。

我期待的是这样一个界面：可以构建关卡及其组成部分（精灵、实体、图块布局），然后点击播放，实际游玩这个关卡。我先打开了单 Agent 运行的产出，最初看到的应用似乎符合预期。

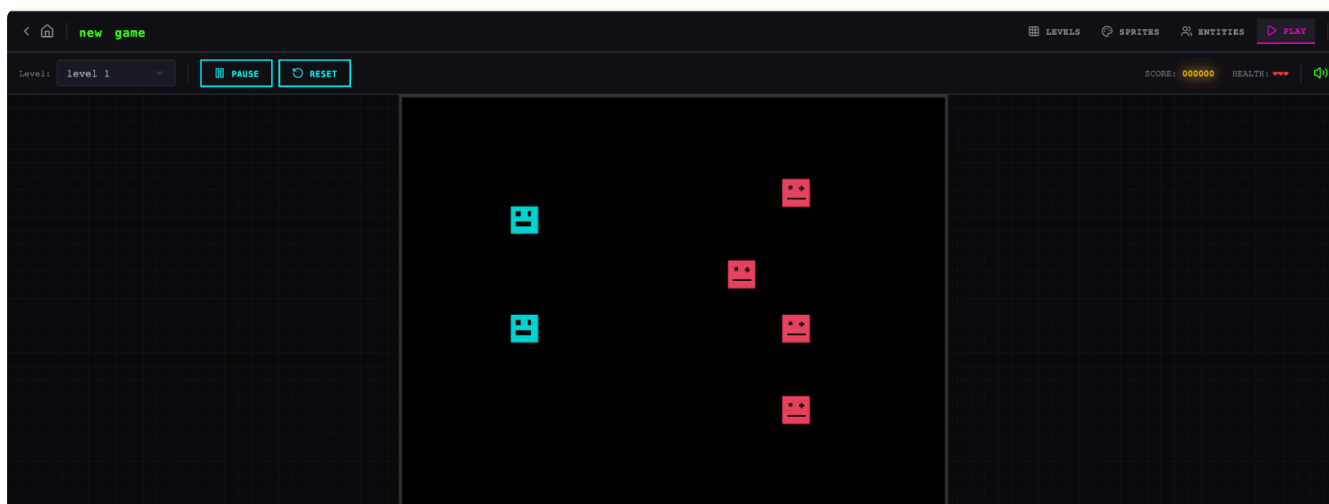
然而，随着我逐步点击，问题开始显现。布局浪费了空间，固定高度的面板让视口大部分区域空空如也。工作流程也很僵硬。尝试向关卡添加内容时，应用提示我先创建精灵和实体，但界面上没有任何引导告诉我应该按照这个顺序操作。更关键的是，游戏本身坏了。实体出现在屏幕上，却对输入毫无反应。深入查看代码后发现，实体定义与游戏运行时之间的连接出了问题，界面上却完全没有指示问题出在哪里。



Initial screen when opening the app created by the solo harness.



Creating a sprite in the sprite editor made by the solo harness



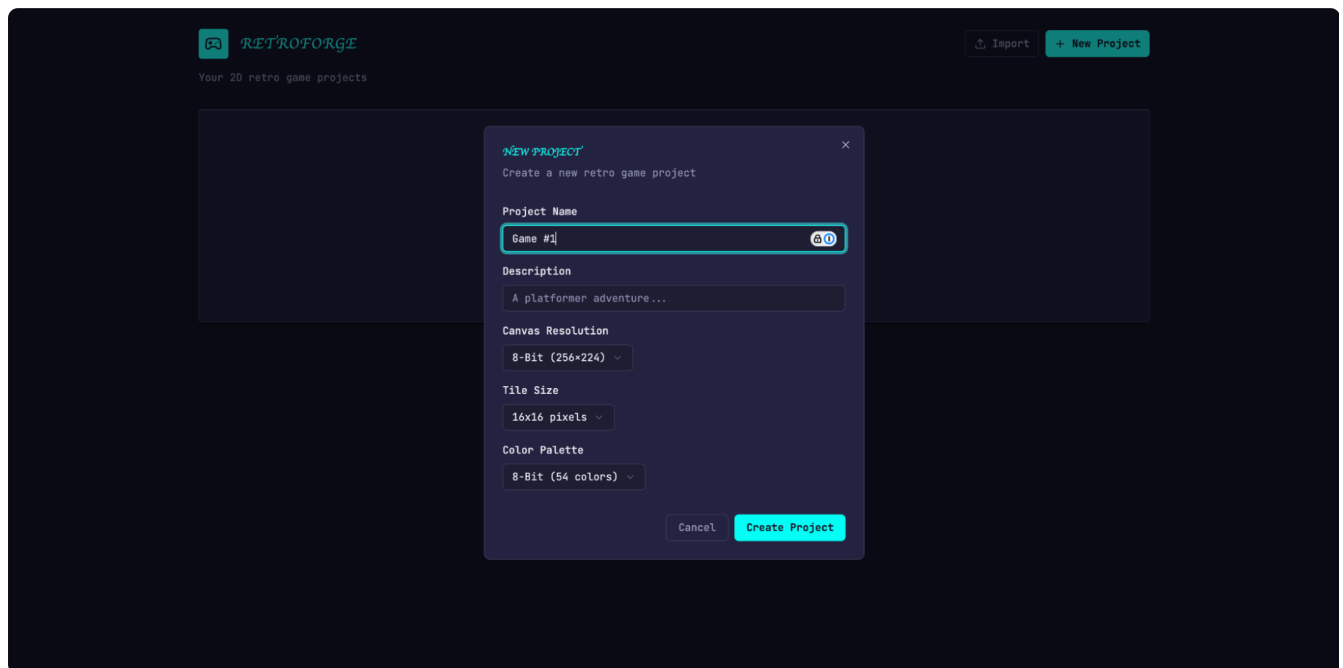
Trying unsuccessfully to play the level I created

评估完单 Agent 运行后，我转向完整框架的运行结果。它从同一句提示开始，但规划器将其扩展成一份包含 16 项功能、分布在十个迭代周期中的规格，远远超出了单 Agent 运行尝试的范围。除了核心编辑器和游玩模式，规格还要求精灵动画系统、行为模板、音效和音乐、AI 辅助精灵生成器和关卡设计器，以及带有可分享链接的游戏导出功能。我允许规划器访问我们的前端设计技能；它读取后，为应用制定了一套视觉设计语言，作为规格的一部分。在每个迭代周期中，生成器和评估器都会协商一份契约，定义该轮的具体实现细节，以及用于验证完成情况的可测试行为。

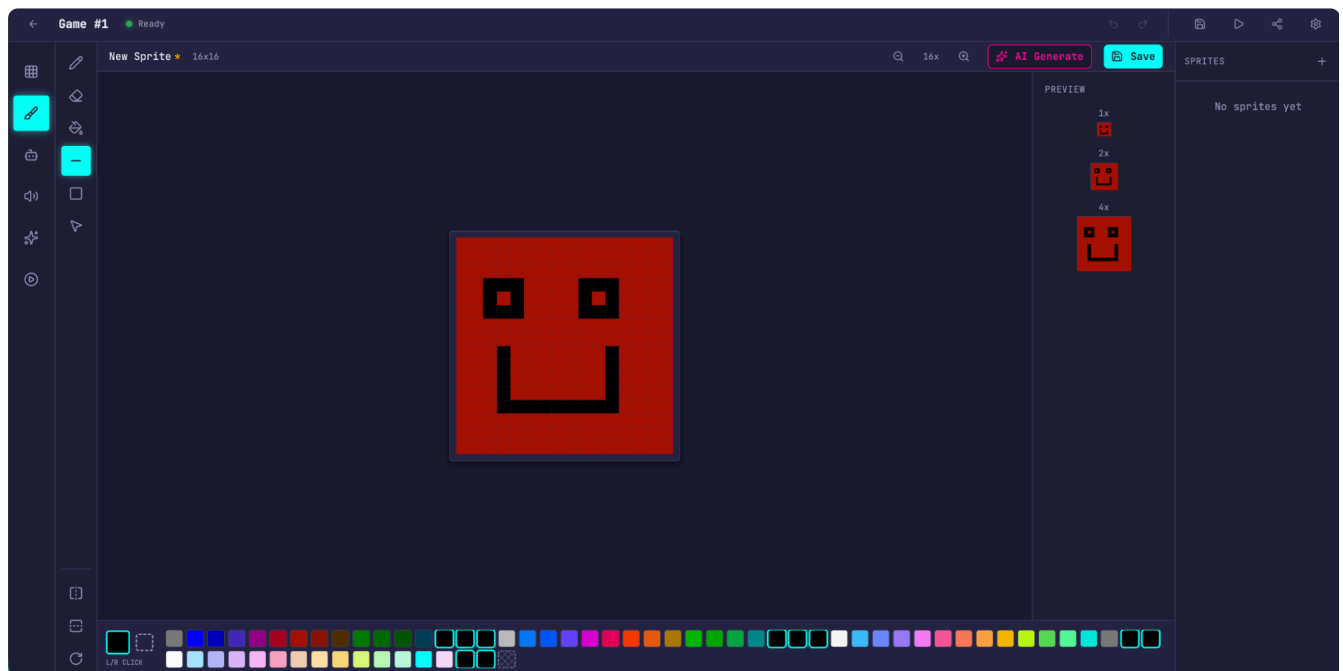
这个应用一开始就比单 Agent 的产出更精致、更流畅。画布充分利用了整个视口，面板尺寸合理，界面具有一致的视觉风格，也遵循了规格中的设计方向。不过，单 Agent 版本的一些笨拙之处仍然存在：工作流程依然没有明确说明，应先创建精灵和实体，再尝试向关卡填充内容，我还是得自己摸索。这更像是基础模型产品直觉上的缺口，而非这个运行框架原本要解决的问题。不过，它也表明，在框架内进行有针对性的迭代，有望进一步提升输出质量。

随着我逐一使用这些编辑器，新运行相较于单 Agent 的优势愈发明显。精灵编辑器更加丰富、功能更加完整，工具面板更清晰、颜色选择器更好用，缩放控件也更实用。

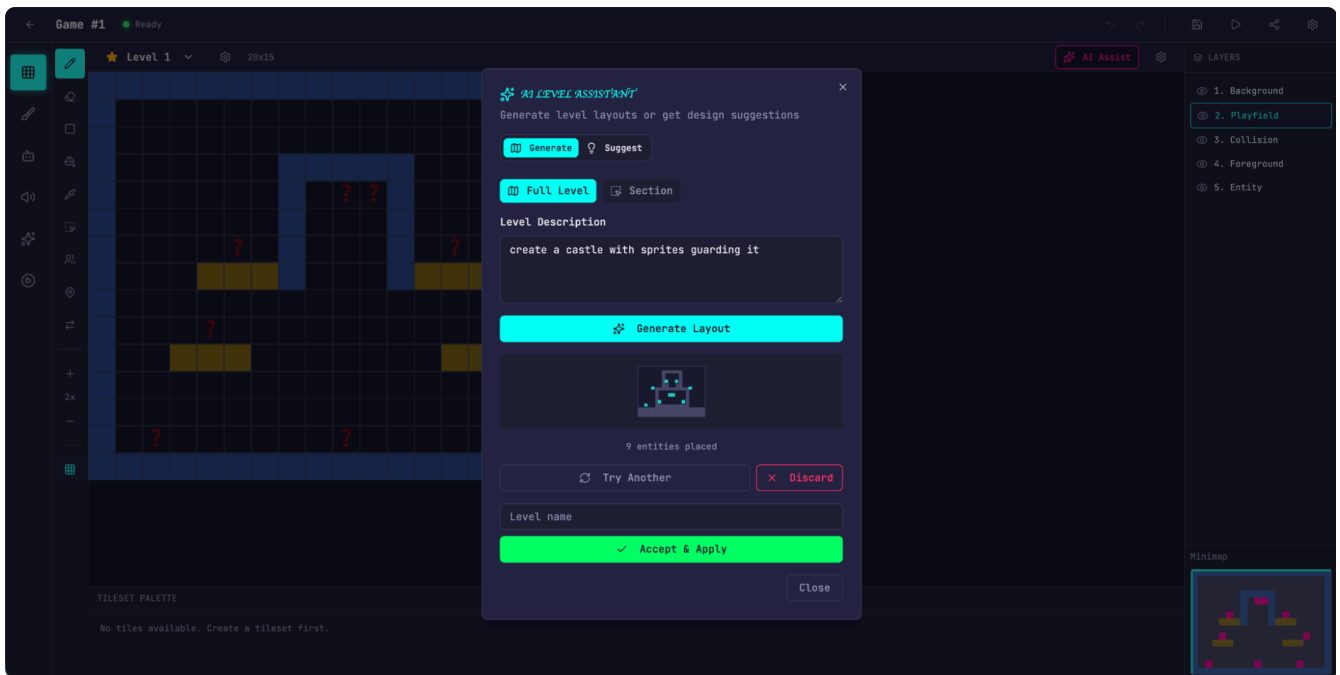
由于我要求规划器在规格中融入 AI 功能，应用还内置了 Claude 集成，让我可以通过提示来生成游戏的不同部分。这显著加快了工作流程。



Initial screen: Creating a new game, in the app built with the full harness



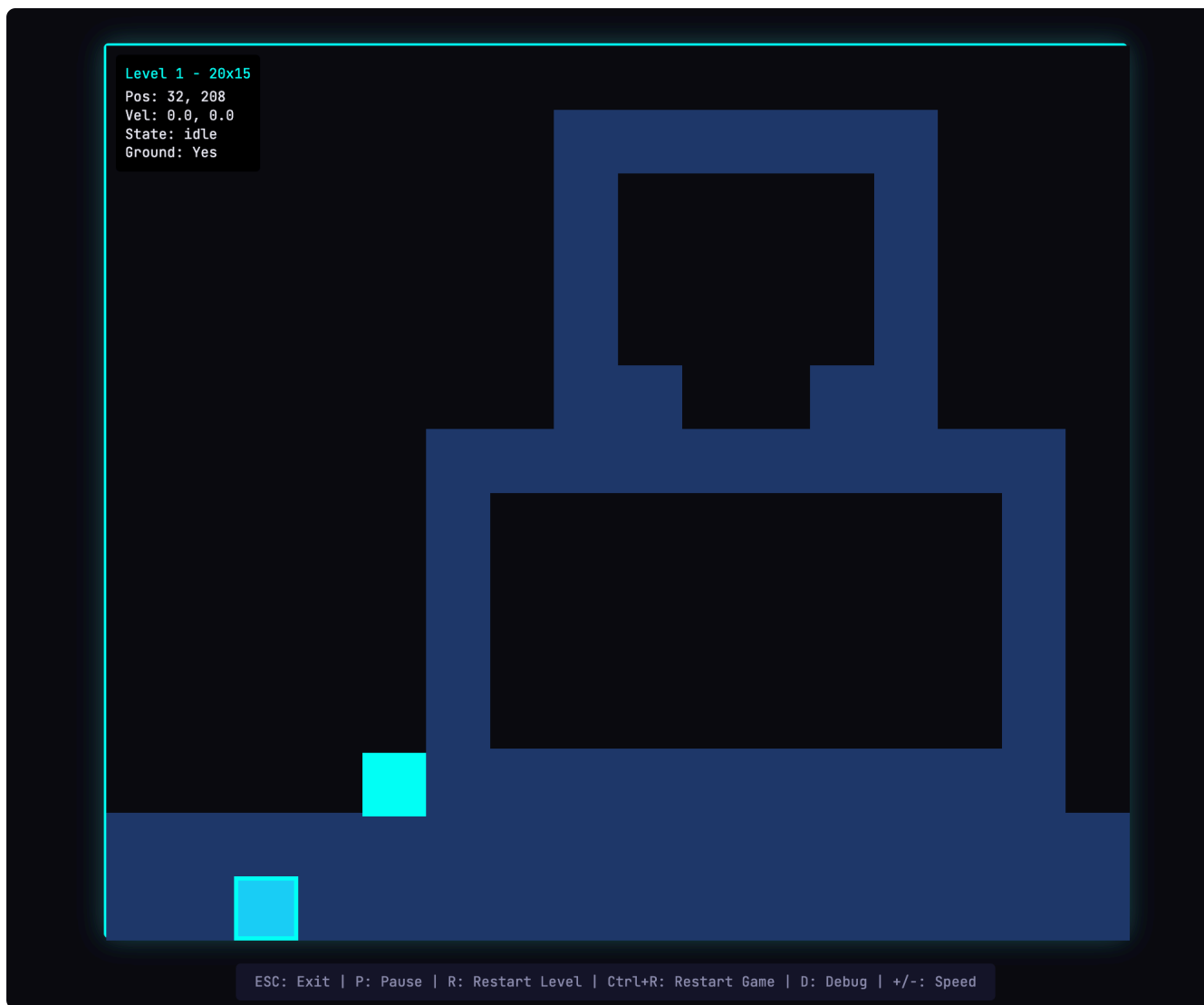
The sprite editor felt cleaner and easier to use



Using the built in AI feature to generate the level



Using the built in AI feature to generate the level



Playing the game I generated

最大的区别在游玩模式。我确实能移动实体并玩这个游戏了。物理效果仍有一些粗糙之处：角色跳上平台后却与平台重叠，直觉上不对劲，但核心功能已经可以使用，而单 Agent 版本没能做到。四处移动一会儿后，我确实碰到了 AI 构建关卡的局限：有一堵大墙跳不过去，我被困住了。这表明，运行框架还可以处理一些常识性改进和边界情况，进一步打磨应用。

阅读日志后，可以清楚看到评估器让实现始终符合规格。每个迭代周期，它都会逐项检查迭代契约中的测试标准，并通过 Playwright 操作运行中的应用，对任何偏离预期行为的情况提交缺陷报告。契约非常细致，仅第 3 个迭代周期就有 27 项覆盖关卡编辑器的标准；评估器发现的问题也足够具体，无需额外调查就能着手修复。下表列出了评估器识别出的几个问题示例：

Contract criterion	Evaluator finding
Rectangle fill tool allows click-drag to fill a rectangular area with selected tile	FAIL — Tool only places tiles at drag start/end points instead of filling the region. <code>fillRectangle</code> function exists but isn't triggered properly on <code>mouseUp</code> .
User can select and delete placed entity spawn points	FAIL — Delete key handler at <code>LevelEditor.tsx:892</code> requires both <code>selection</code> and <code>selectedEntityId</code> to be set, but clicking an entity only sets <code>selectedEntityId</code> . Condition should be <code>selection (selectedEntityId && activeLayer === 'entity')</code> .
User can reorder animation frames via API	FAIL — <code>PUT /frames/reorder</code> route defined after <code>/frame_id</code> routes. FastAPI matches <code>'reorder'</code> as a <code>frame_id</code> integer and returns 422: "unable to parse string as an integer."

让评估器达到这个水平，需要一番功夫。开箱即用的 Claude 并不是一个优秀的质量保证 Agent。早期运行时，我看到它识别出确实存在的问题后，又说服自己这些问题无关紧要，最终仍然批准了工作。它还倾向于只做表面测试，而不是深入探查边界情况，因此更隐蔽的缺陷经常漏网。我的调优循环是：阅读评估器日志，找出它的判断与我不同的案例，再更新质量保证 Agent 的提示词来解决这些问题。经过数轮这样的开发循环，评估器才开始以我认为合理的方式评分。即使如此，运行框架的产出仍显示了模型质量保证能力的局限：细小的布局问题、某些不直观的交互，以及更深层功能中尚未发现的缺陷——评估器没有充分测试这些功能。进一步调优显然还能挖掘更多验证能力。但与核心功能根本无法工作的单 Agent 版本相比，提升是明显的。

迭代运行框架

第一批运行框架结果令人鼓舞，但框架也很臃肿、缓慢且昂贵。顺理成章的下一步，是寻找在不降低表现的前提下简化框架的方法。这一方面出于常识，另一方面也源自一条更普遍的原则：运行框架中的每个组件，都包含着对“模型独自做不到什么”的一种假设。这些假设值得接受严格检验，因为它们可能本来就不正确，也可能随着模型进步而迅速过时。我们的博客文章《构建高效的 Agent》将这一基本思想概括为“找到尽可能简单的解决方案，只有必要时才增加复杂度”。任何维护 Agent 运行框架的人，都会反复遇到这种模式。

第一次尝试简化时，我大幅削减了框架，并尝试了几个有创意的新想法，但没能复现原版的表现。同时，也变得很难判断运行框架设计中的哪些部分真正起到了关键支撑作用，以及具体如何发挥作用。基于这次经验，我改用更系统的方法：每次只移除一个组件，再检查它对最终结果造成了什么影响。

在这些迭代期间，我们还发布了 Opus 4.6，这进一步推动我降低运行框架的复杂度。有充分理由预期，4.6 所需的外部辅助结构会比 4.5 更少。我们的发布博客写道：“[Opus 4.6] 规划更加审慎，能够更长时间地持续执行 Agent 任务，在大型代码库中运行得更加可靠，并且拥有更好的代码审查和调试能力，能够发现自己的错误。”它在长上下文检索方面也有显著提升。这些能力，正是我们构建运行框架时一直试图补足的。

移除迭代周期结构

我先彻底移除了迭代周期结构。这个结构此前帮助模型将工作拆成小块，以连贯地开展任务。考虑到 Opus 4.6 的进步，我们有充分理由相信，即使没有这种拆分，模型本身也能胜任工作。

我保留了规划器和评估器，因为二者仍然带来明显价值。没有规划器时，生成器会把范围定得过小：面对原始提示，它会直接开始构建，而不会先制定规格，最终输出的应用功能也不如经过规划器的版本丰富。

移除迭代周期结构后，我将评估器改为在整个运行结束时统一评估，而不再逐轮评分。由于模型能力强了很多，评估器在某些运行中的关键程度也发生了变化；它是否有用，取决于任务相对于模型独立可靠完成能力边界的位置。对 4.5 来说，这个边界离得很近：我们的构建任务正处于生成器独立做好工作的能力边缘，而评估器能在整个构建过程中发现重要问题。到了 4.6，模型的原生能力增强，边界也向外扩展。过去需要评估器检查才能连贯实现的任务，现在往往已经在生成器独立胜任的范围内；对于这个边界内的任务，评估器变成了不必要的开销。但对于仍处于生成器能力边缘的构建部分，评估器依然能带来实际提升。

实践上的含义是：是否使用评估器，并不是一个固定不变的是非选择。当任务超出当前模型独立可靠完成的范围时，评估器的成本才值得付出。

在简化结构的同时，我还增加了提示，以改善运行框架在各个应用中构建 AI 功能的方式，具体来说，是让生成器构建一个能够通过工具驱动应用自身功能的真正 Agent。这确实需要反复迭代，因为相关知识足够新，在 Claude 的训练数据中覆盖很少。但经过足够调优后，生成器已经能够正确构建 Agent。

更新后运行框架的结果

为了测试更新后的框架，我使用下面的提示生成一个数字音频工作站 (DAW)，即用于作曲、录音和混音的音乐制作程序：

使用 Web Audio API，在浏览器中构建一个功能完整的 DAW。

这次运行依然漫长而昂贵，耗时约 4 小时，token 成本为 124 美元。

大部分时间花在构建 Agent 上；它无需 Opus 4.5 曾依赖的迭代周期拆分，就连贯地工作了两个多小时。

Agent & Phase	Duration	Cost
Planner	4.7 min	\$0.46
Build (Round 1)	2 hr 7 min	\$71.08
QA (Round 1)	8.8 min	\$3.24
Build (Round 2)	1 hr 2 min	\$36.89
QA (Round 2)	6.8 min	\$3.09
Build (Round 3)	10.9 min	\$5.88
QA (Round 3)	9.6 min	\$4.06
Total V2 Harness	3 hr 50 min	\$124.70

与之前的框架一样，规划器将这一行提示扩展成完整规格。从日志中可以看到，生成器模型在规划应用与 Agent 设计、接入 Agent，以及交给质量保证环节前进行测试这些方面，都做得很好。

即便如此，质量保证 Agent 仍然发现了实质性的缺漏。在第一轮反馈中，它指出：

这是一个很强的应用，设计还原度出色，AI Agent 扎实，后端也不错。主要不合格项是功能完整性：虽然应用看起来令人印象深刻，AI 集成也运转良好，但有几项核心 DAW 功能只是展示，缺乏深入交

互：片段无法在时间线上拖动或移动，没有乐器界面面板（合成器旋钮、打击垫），也没有可视化效果编辑器（均衡器曲线、压缩器电平表）。这些不是边界情况，而是让 DAW 能够实际使用的核心交互，而且规格中明确要求了它们。

在第二轮反馈中，它再次发现了几处功能缺口：

剩余缺口：

- 录音仍然只有占位实现（按钮可以切换状态，但没有采集麦克风音频）
- 尚未实现通过拖动边缘调整片段长度，以及拆分片段
- 效果可视化只有数值滑块，没有图形界面（没有均衡器曲线）

当生成器独自工作时，仍然容易遗漏细节，或只做功能的占位实现。质量保证 Agent 能发现这些最后一公里的问题并交给生成器修复，因此依然具有价值。

根据提示，我期待的是一个可以创建旋律、和声与鼓点型，将它们编排成歌曲，并在过程中获得内置 Agent 帮助的程序。下面的视频展示了结果。

Video / 视频（联网播放）

这个应用距离专业音乐制作程序还很远，Agent 的作曲能力显然也有很大的提升空间。此外，Claude 实际上听不到声音，这让质量保证反馈循环在音乐品味方面的作用有所减弱。

不过，最终应用已经具备一个可用音乐制作程序的所有核心组成部分：在浏览器中运行的编排视图、混音器和播放控制功能。除此之外，我还能完全通过提示制作出一段短歌：Agent 设置速度和调性、写下旋律、创建鼓轨、调整混音电平，并添加混响。作曲所需的核心基础操作都已具备，Agent 能自主驱动它们，通过工具从头到尾完成一段简单作品。你可以说，它还没有做到“音准完美”，但正朝着这个方向前进。

接下来会怎样

随着模型不断进步，我们大体可以预期，它们将能够工作更长时间，处理更复杂的任务。在某些情况下，这意味着模型周围的辅助结构会逐渐变得不那么重要；开发者可以等待下一代模型，看到某些问题自行消失。另一方面，模型越好，就越有空间开发运行框架，完成超出模型基线能力的复杂任务。

因此，这项工作中有几条经验值得延续。围绕你正在使用的模型开展实验、阅读它处理真实问题时的执行轨迹，并调优表现以达到期望结果，始终是好做法。在处理更复杂的任务时，有时可以通过拆分任务、让专门的 Agent 处理问题的各个方面，获得额外提升。新模型发布后，通常应重新审视运行框架，去掉那些已经不再对表现起关键支撑作用的部分，并加入新的部分，以获得此前可能无法实现的更强能力。

这项工作让我确信：随着模型进步，值得探索的运行框架组合空间并不会缩小，而是会发生转移。AI 工程师最有趣的工作，就是不断寻找下一个新颖组合。

致谢

特别感谢 Mike Krieger、Michael Agaby、Justin Young、Jeremy Hadfield、David Hershey、Julius Tarng、Xiaoyi Zhang、Barry Zhang、Orowa Sidker、Michael Tingley、Ibrahim Madha、Martina Long 和 Canyon Robbins 对这项工作的贡献。

也感谢 Jake Eaton、Alyssa Leonard 和 Stef Sequeira 帮助完善本文。

附录

规划器 Agent 生成的计划示例。

RetroForge - 2D Retro Game Maker

Overview

RetroForge is a web-based creative studio for designing and building 2D retro-style video games. It combines the nostalgic charm of classic 8-bit and 16-bit game aesthetics with modern, intuitive editing tools—enabling anyone from hobbyist creators to indie developers to bring their game ideas to life without writing traditional code.

The platform provides four integrated creative modules: a tile-based Level Editor for designing game worlds, a pixel-art Sprite Editor for crafting visual assets, a visual Entity Behavior system for defining game logic, and an instant Playable Test Mode for real-time gameplay testing. By weaving AI assistance throughout (powered by Claude), RetroForge accelerates the creative process—helping users generate sprites, design levels, and configure behaviors through natural language interaction.

RetroForge targets creators who love retro gaming aesthetics but want modern conveniences. Whether recreating the platformers, RPGs, or action games of their childhood, or inventing entirely new experiences within retro constraints, users can prototype rapidly, iterate visually, and share their creations with others.

Features

1. Project Dashboard & Management

The Project Dashboard is the home base for all creative work in RetroForge. Users need a clear, organized way to manage their game projects—creating new ones, returning to works-in-progress, and understanding what each project contains at a glance.

User Stories: As a user, I want to:

- Create a new game project with a name and description, so that I can begin designing my game
- See all my existing projects displayed as visual cards showing the project name, last modified date, and a thumbnail preview, so that I can quickly find and continue my work
- Open any project to enter the full game editor workspace, so that I can work on my game
- Delete projects I no longer need, with a confirmation dialog to prevent accidents, so that I can keep my workspace organized
- Duplicate an existing project as a starting point for a new game, so that I can reuse my previous work

Project Data Model: Each project contains:

Project metadata (name, description, created/modified timestamps)
Canvas settings (resolution: e.g., 256x224, 320x240, or 160x144)
Tile size configuration (8x8, 16x16, or 32x32 pixels)
Color palette selection
All associated sprites, tilesets, levels, and entity definitions

...